

Dynamic String Buffer in C

Implementation demonstrating automatic growth using realloc, safe memory management, and destructor cleanup.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *data;
    size_t length;
    size_t capacity;
} StringBuffer;

/* Constructor */
StringBuffer *sb_init(size_t initial_capacity) {
    StringBuffer *sb = (StringBuffer *)malloc(sizeof(StringBuffer));
    if (sb == NULL) {
        return NULL;
    }

    sb->data = (char *)malloc(initial_capacity);
    if (sb->data == NULL) {
        free(sb);
        return NULL;
    }

    sb->length = 0;
    sb->capacity = initial_capacity;
    sb->data[0] = '\0';

    return sb;
}

/* Append string, growing buffer as needed */
int sb_append(StringBuffer *sb, const char *str) {
    if (sb == NULL || str == NULL) {
        return 0;
    }

    size_t str_len = strlen(str);
    size_t required = sb->length + str_len + 1;

    while (required > sb->capacity) {
        size_t new_capacity = sb->capacity * 2;

        char *temp = (char *)realloc(sb->data, new_capacity);
        if (temp == NULL) {
            return 0; /* realloc failed, original pointer still valid */
        }

        sb->data = temp;
        sb->capacity = new_capacity;

        printf("Buffer grew to capacity: %zu bytes\n", sb->capacity);
    }

    memcpy(sb->data + sb->length, str, str_len + 1);
    sb->length += str_len;

    return 1;
}

/* Destructor */
void sb_free(StringBuffer *sb) {
    if (sb != NULL) {
        free(sb->data);
        free(sb);
    }
}

int main(void) {
    StringBuffer *sb = sb_init(8);
    if (sb == NULL) {
```

```
        fprintf(stderr, "Failed to initialize StringBuffer\n");
        return 1;
    }

    printf("Initial capacity: %zu bytes\n", sb->capacity);

    sb_append(sb, "Hello");
    printf("String: %s\n", sb->data);

    sb_append(sb, ", Dynamic ");
    printf("String: %s\n", sb->data);

    sb_append(sb, "String Buffer Example!");
    printf("String: %s\n", sb->data);

    printf("Final length: %zu\n", sb->length);
    printf("Final capacity: %zu\n", sb->capacity);

    sb_free(sb);
    return 0;
}
```